Table of Contents

1. Project Description

Mythic Gates is a full-stack fantasy role-playing web application where players collect unique characters, battle powerful bosses, and earn rewards to expand their roster. The project is designed to demonstrate modern software engineering practices by combining a secure backend, responsive frontend, and engaging game mechanics into a complete, production-style application.

The application is built using Spring Boot for the backend, Angular for the frontend, and PostgreSQL as the database. Authentication is implemented using JWT access tokens with HTTP-only refresh tokens, providing a secure and seamless login experience. Cloud-hosted assets like images are managed through Cloudinary, allowing character and boss images to be stored and delivered efficiently.

Players begin by creating an account and receiving a set of starter characters. Additional characters can be unlocked using in-game gold earned through gameplay. Each character possesses unique statistics, abilities, mana costs, and skill cooldowns, encouraging strategic decision-making during battles. Players face a variety of fantasy bosses, each with distinct combat attributes like critical hit chances, dodge mechanics, healing abilities, and reward values.

The battle system follows a turn-based combat model where players choose skills, manage health and mana, and react to boss attacks. Features such as critical hits, dodge chances, healing, mana restoration, and skill cooldowns provide tactical depth while maintaining an accessible gameplay experience. Every battle is recorded, allowing players to review their battle history and continue unfinished encounters.

The application includes a user interface featuring a character roster, character details, battle arena, battle history, authentication pages, dashboard, and in-game

shop. Toast notifications, pagination, secure route protection, and automatic token refresh contribute to a polished user experience.

From a software engineering perspective, Mythic Gates emphasizes clean architecture and maintainability. The backend follows a layered architecture with controllers, services, repositories, DTOs, mappers, and centralized exception handling. RESTful APIs are used for communication between the frontend and backend, while data validation, consistent API responses, and role-based authorization ensure reliability and security throughout the application.

Overall, Mythic Gates serves as both an enjoyable fantasy RPG experience and a comprehensive demonstration of full-stack application development, showcasing secure authentication, database design, REST API development, responsive frontend engineering, and modern software architecture principles.

2. Project Requirements

2.1. Core Features

1. User Registration and Authentication

- User should be able to create accounts and securely login to the platform.
- Implement JWT for user authentication
- Role-based access control (USER / ADMIN)
- Regenerate access token using refresh token upon JWT expiry and provide seamless user experience without logging out the user.

2. User System

- Upon successful registration, give users 3 free starter characters.
- Track and display user gold balance and total number of bosses defeated.
- Enable option for user to logout.

3. Character System

- Display a list of characters owned by the authenticated user.
- Each character should have a rarity (COMMON / RARE / EPIC / LEGENDARY / MYTHIC), base stats (hp, mana, attack, defense, heal power, crit change, dodge chance and an image).
- Each character has 4 skills (2 basic, 2 special)
- Implement pagination to navigate between different pages of the character roster.
- **Admin** can add a new character, update its stats, and delete it if needed.
- **User** can unlock a new character by spending their gold.
- Implement detail view of a character along with the skills they own.

4. Skill System

- Each skill is bound to a specific character
- Each skill has damage multiplier, mana cost, cooldown
- Each character can have 2 basic skills and 2 special skills
- Each skill has a cooldown
- **Admin** can add skills for a character in bulk

5. Boss System

- **Users** should be able to view the list of bosses in the game.
- **Admin** should be able to add the bosses in the game
- Bosses should have mechanics like:
    - Critical Hit
    - Dodge change
    - Rage mode
    - Periodic healing every N turn

6. Battle Engine

- Actual place of battle
- Implement character and boss selection
- Allow random character and random boss selection for battle
- Character attacks boss and uses mana
- Bosses attacks character
- Character can choose among 4 skills to attack
- Each attack can have cooldown before using it again.
- When mana is finished, attack must not land.
- Restore mana feature to fully restore the mana
- Heal feature to heal a certain amount and consume mana (minimum 20 mana)
- Battle log for each turn

- When a user disconnects from a battle, store the battle state and allow continuing the battle later on
- If a battle is in continue state, restrict from starting a new game
- User can have an option to forfeit battle

7. Economy System

- Add gold upon defeating a boss
- Upon purchase of new character, deduct gold

8. Store System

- Display a catalog of characters that are unlocked and locked
- Allow user to use gold to unlock a character
- User can view the skills and the character statistics from the store itself

2.2. Non-functional requirements

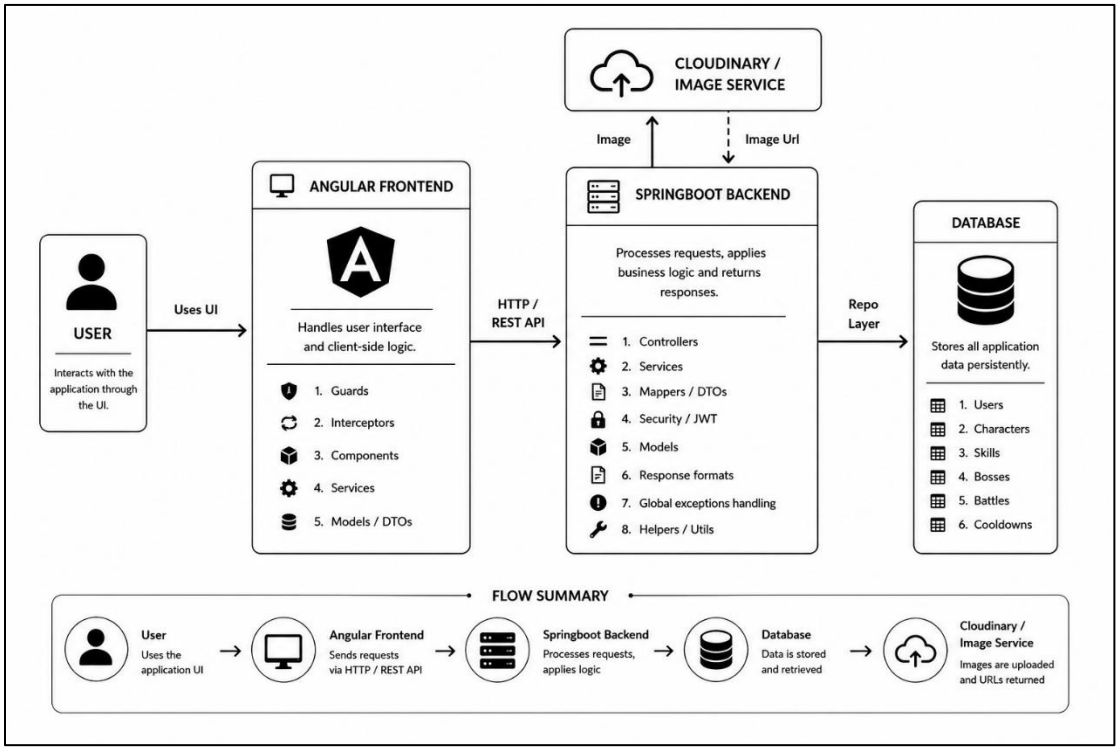
1. Security and Privacy

- The application must employ industry-standard security measures to protect user data

2. User-Friendly Interface

- The application should have an intuitive and visually appealing user interface matching an ancient battle theme.

3. System Architecture



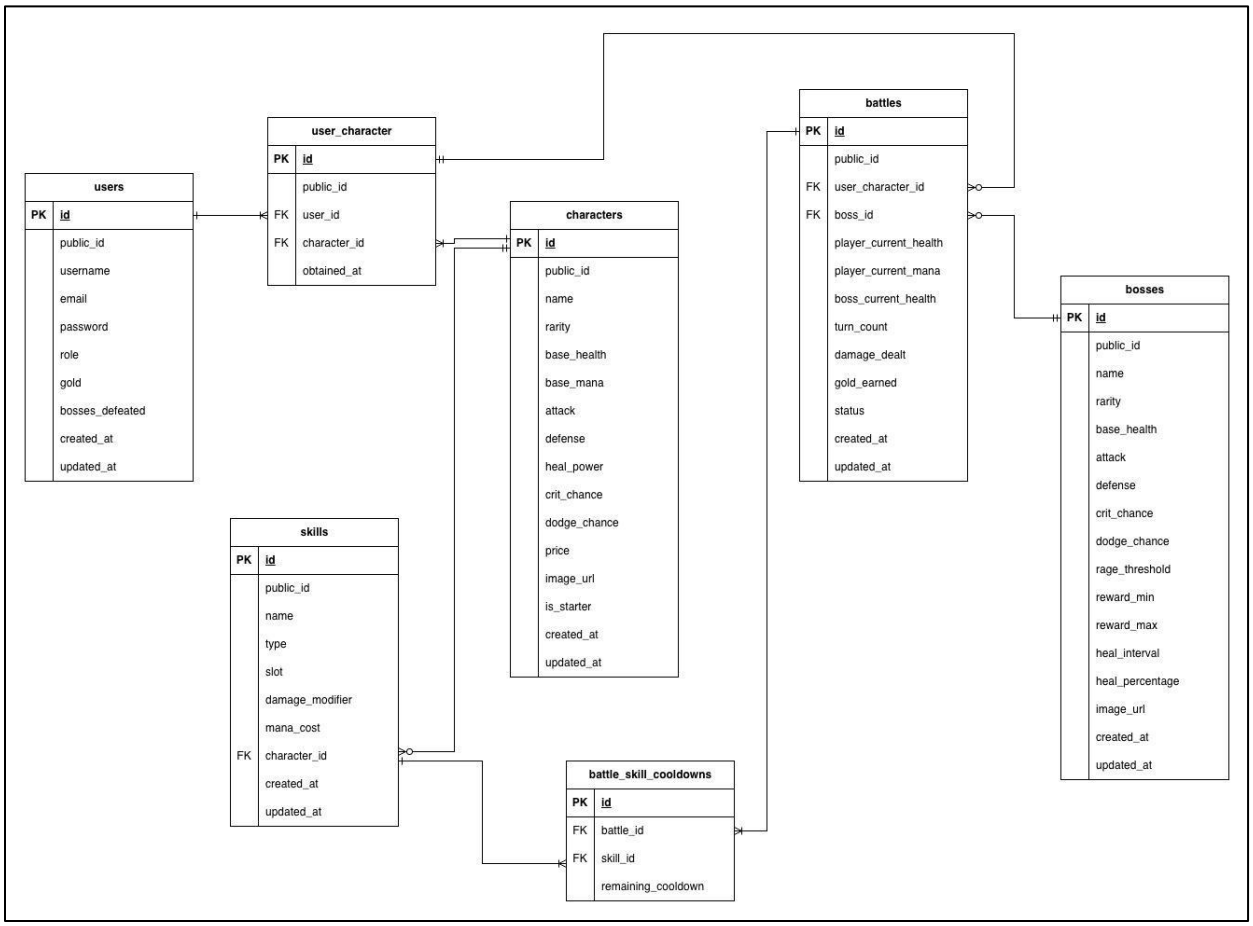
The system architecture of Mythic Gates follows a client-server model. The user interacts with the application through an Angular frontend running in the browser. The frontend communicates with the Spring Boot backend using RESTful API requests.

The backend is responsible for handling authentication, authorization, battle logic, character management, boss management, skill management, and user progression. Controllers receive incoming requests and pass them to the service layer, where the core business logic is processed. The repository layer uses Spring Data JPA to communicate with the PostgreSQL database.

The PostgreSQL database stores persistent application data such as users, characters, skills, bosses, owned characters, battles, and battle skill cooldowns. Cloudinary is used as an external image storage service for character and boss images.

Security is handled using JWT access tokens and HTTP-only refresh tokens. Protected endpoints require valid authentication, while admin-only features are restricted using role-based authorization.

4. Database Design (ERD)



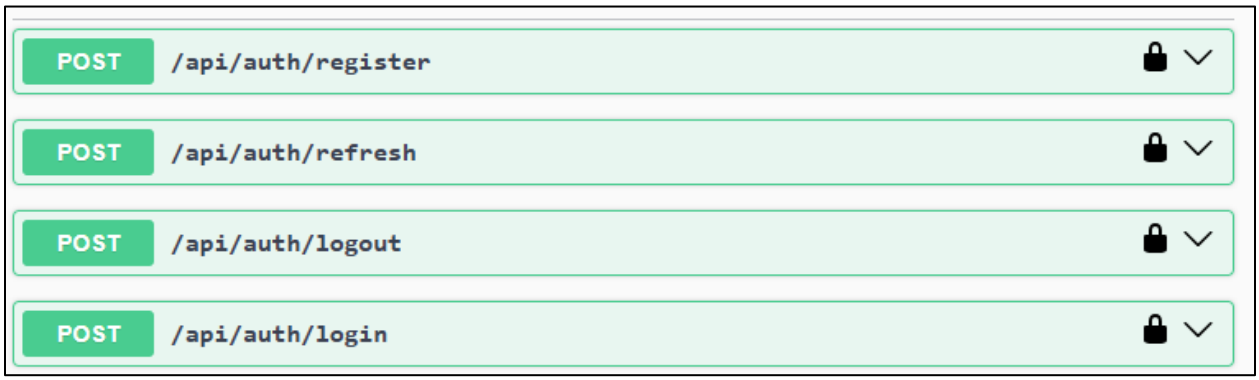
Relationships:

- One User can own many UserCharacters.
- Each UserCharacter belongs to exactly one User.
- One Character can be unlocked by many UserCharacters.
- Each UserCharacter represents exactly one Character.
- One Character can have many Skills.
- Each Skill belongs to exactly one Character.
- One UserCharacter can participate in many Battles.
- Each Battle uses exactly one UserCharacter.
- One Boss can be fought in many Battles.

- Each Battle contains exactly one Boss.
- One Battle can have many BattleSkillCooldown records.
- Each BattleSkillCooldown belongs to exactly one Battle.
- One Skill can appear in many BattleSkillCooldown records.
- Each BattleSkillCooldown tracks exactly one Skill.

5. API Overview

5.1. Auth Endpoints

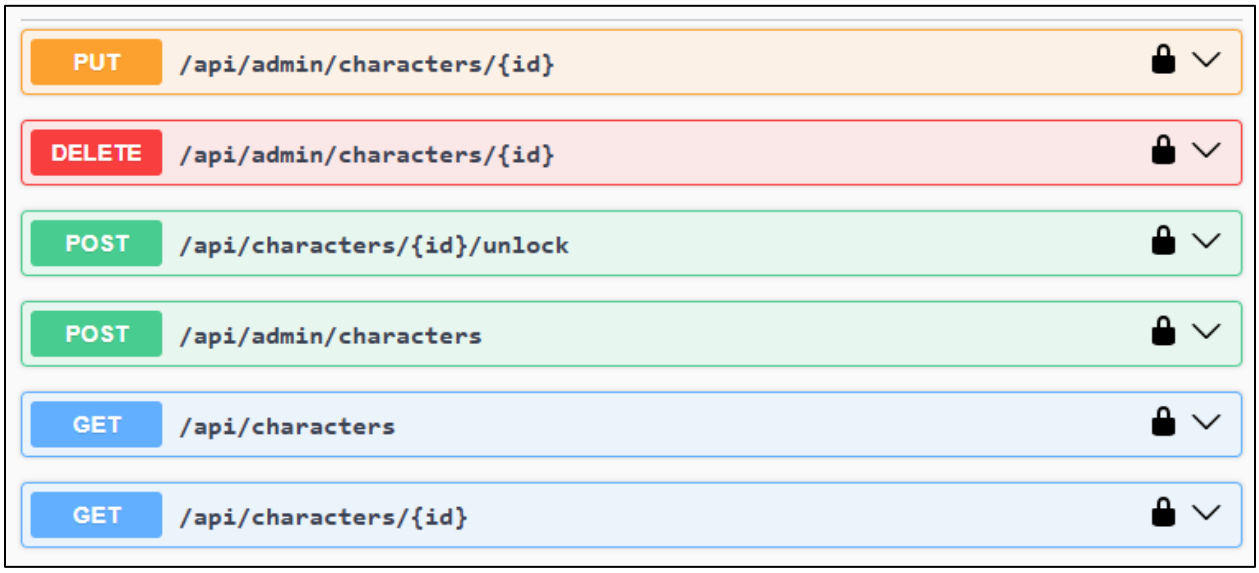


Method	Endpoint	Description
POST	/api/auth/register	Registers a new user
POST	/api/auth/login	Authenticate user and return JWT
POST	/api/auth/refresh	Generate a new access token
POST	/api/auth/logout	Logout and invalidate refresh cookie

5.2. User Endpoints

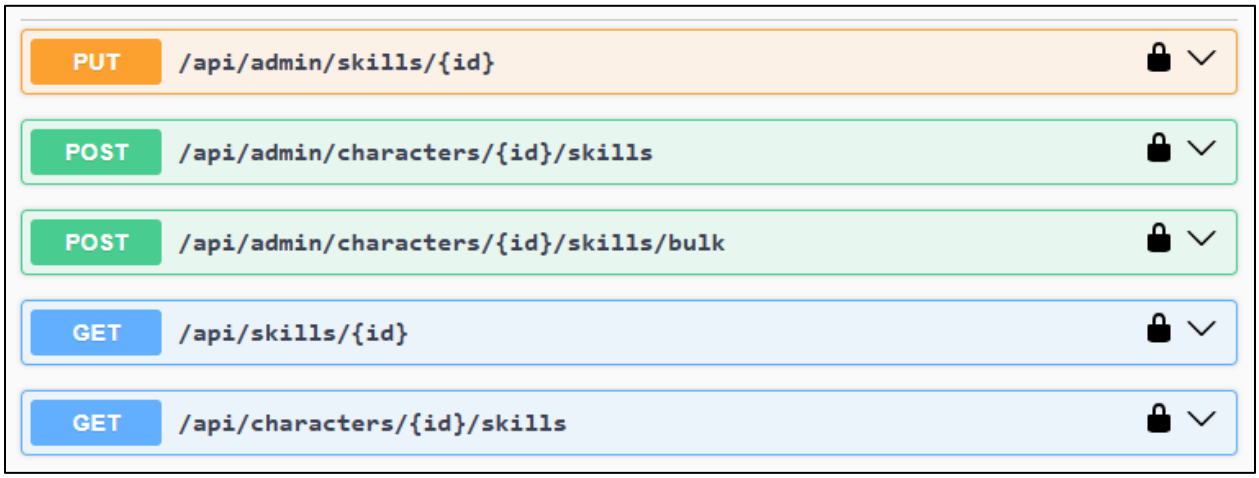
Method	Endpoint	Description
POST	/api/users/deduct/gold	Deduct user balance in-game
POST	/api/users/add/gold	Add amount to user balance in-game
GET	/api/users/me	Retrieve the current authenticated user
GET	/api/users/me/characters	Retrieve the characters owned by the user
GET	/api/users/balance	Retrieve the current balance for the user

5.3. Character Endpoints



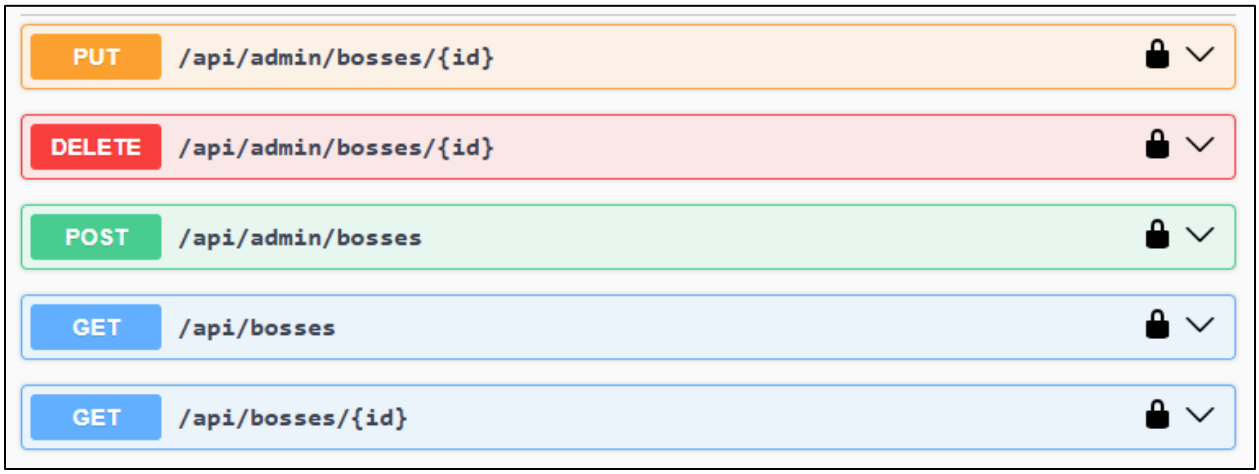
Method	Endpoint	Description
PUT	/api/admin/characters/{id}	Update an existing character detail
DELETE	/api/admin/characters/{id}	Delete an existing character
POST	/api/characters/{id}/unlock	Unlock a character by spending user balance
POST	/api/admin/characters	Add a new character
GET	/api/characters	Retrieve all characters
GET	/api/characters/{id}	Retrieve a specific character

5.4. Skill Endpoints



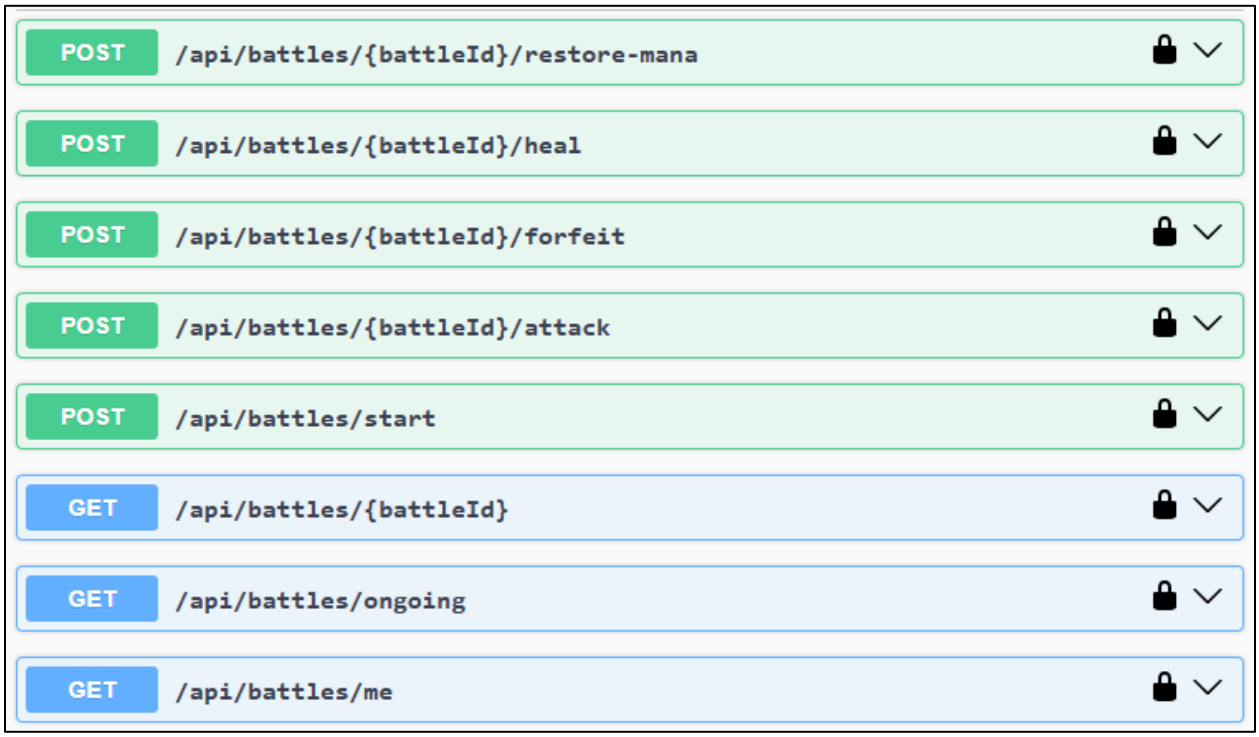
Method	Endpoint	Description
PUT	/api/admin/skills/{id}	Update a skill detail
POST	/api/admin/characters/{id}/skills	Add a new skill to the character
POST	/api/admin/characters/{id}/skills/bulk	Add all the skills for a specific character at once
GET	/api/skills/{id}	Retrieve a specific skill
GET	/api/characters/{id}/skills	Retrieve all the skill for a character

5.5. Boss Endpoints



Method	Endpoint	Description
PUT	/api/admin/bosses/{id}	Update a boss detail
DELETE	/api/admin/bosses/{id}	Delete a specific boss
POST	/api/admin/bosses	Add a new boss
GET	/api/bosses	Retrieve all the bosses
GET	/api/bosses/{id}	Retrieve a specific boss

5.6. Battle Endpoints



Method	Endpoint	Description
POST	/api/battles/{battleId}/restore-mana	Restore the mana of a character in battle fully
POST	/api/battles/{battleId}/heal	Restore the health of a character in battle by specific amount
POST	/api/battles/{battleId}/forfeit	End the current ongoing battle and set it as LOST
POST	/api/battles/{battleId}/attack	Deal damage to a boss using an attack skill
POST	/api/battles/start	Start a battle game
GET	/api/battles/{battleId}	Retrieve a battle detail
GET	/api/battles/ongoing	Retrieve the ongoing battle
GET	/api/battles/me	Retrieve all the user battle for the authenticated user

6. User Manual

6.1. User Registration

- Open the application and click on register
- Enter your details:
    - Username
    - Email
    - Password
- Click Create Account

The system will create the account and assign three starter characters. The user is then redirect to the login page.

6.2. Login

- Open the login page
- Enter username and password
- Clock Login

After successful authentication, the JWT access token is stored by the frontend, the refresh token is stored as an HTTP-only cookie, and the user is redirected to the game menu.

6.3. Game Menu / Play

- The game menu provides access to:
    - Character Roster
    - Battle History
    - Shop
    - Boss Codex
    - The Game itself
- The game menu has two options:
    - Enter Arena (Start a new battle)

    - Return to Fight (Continue the previous ongoing fight)
- Starting a new game requires choosing of a character and a boss. The characters and bosses can be randomized using the provided randomize button.
- Upon entering a battle, a list of attack skills, heal and restore abilities can be found.
- User can forfeit a battle using the close button at the top right of the battle screen.

6.4. Characters (Character roster)

- The roster displays all unlocked characters
- Each character card shows:
    - Character Image
    - Name
    - Rarity
    - Health
    - Mana
    - Attack
    - Defense
- Selecting the View details displays:
    - Full Statistics
    - Skills
    - Character Information

6.5. Bosses (Boss codex)

- Similarly, to the characters, all the bosses in the game are displayed with their statistics. Upon clicking view details, it shows the full statistics of the boss.

6.6. Shop

- The shop displays characters available for purchase and those that are unlocked too.
- Each character includes:
    - Image
    - Name
    - Rarity
    - Gold cost
    - Statistics
- To unlock a character, click on unlock
- If sufficient gold is available, gold is deducted and the character is permanently unlocked

6.7. Starting a battle

- Inside of the game menu, click on “Enter Arena”
- Select one of the unlocked characters
- Choose a boss
- The battle starts after the loading screen
- If an unfinished battle already exists, users will have to continue that battle instead of creating a new one.

6.8. Battle Screen

The battle screen displays in information of:

- Player
    - Current health
    - Current mana
    - Character portrait
- Boss
    - Current health

    - Boss portrait
- Battle Information
    - Turn number
    - Battle log

6.9. Battle Actions

- Attack

  Performs a skill attack against the boss and the system automatically calculates the following:
    - Calculates damage
    - Applies critical hits
    - Checks dodge chance
    - Updates health and mana
    - Triggers the boss turn
- Heal

  Consumes mana to restore a certain amount of health. Healing cannot exceed the character’s maximum health.
- Restore Mana

  Fully restores the player’s mana
- Forfeit

  Ends the current battle. The battle is marked as defeat with no gold earned.
- Battle end

  A battle ends when:
    - The boss is defeated
    - The player is defeated
    - The player forfeits

  The rewards are granted automatically after victory.

6.10. Battle History

- The battle history page displays every completed and ongoing battle. Each record includes:
    - Character
    - Boss
    - Status
    - Damage dealt
    - Turn count
    - Gold earned
    - A button to view the game state after the last turn

6.11. Logout

- Select Logout from the navigation menu. The system will redirect the user to the login page.